Hello, my name is Chinedu Arize, and I'm excited to present my personal portfolio web application, which I built as part of my internship task.

In this presentation, I will walk you through how I structured and developed the entire project using HTML, CSS, and JavaScript. I will explain how each part works, as well as the decisions I made during development.

To begin with, I created a folder on my desktop named "Internship Task." This folder serves as the root directory of my project, meaning it contains all the files and resources needed for the application.

I then opened this folder using Visual Studio Code, which is the code editor I used to write and manage all my code.

Inside this main project folder, I organized my files in a clear and structured way. First, I created multiple HTML files to represent different pages of the website.

The main file is called index.html, which serves as the homepage of the portfolio. This is the first page users see when they visit the website.

Next, I created about.html, which contains detailed information about me, including my background and introduction.

I also created skills.html, where I listed my technical skills such as HTML, CSS, and JavaScript.

Then I created projects.html, which is used to showcase the projects I have worked on, including descriptions and visuals.

In addition, I created resume.html, where I presented my educational background, experience, and other professional details.

Finally, I created contact.html, which allows users or recruiters to reach out to me through contact information or a form.

To handle the design of all these pages, I created a single CSS file named portfolio.css. This file is responsible for styling the entire website, including layout, colors, fonts, spacing, and responsiveness. By using one CSS file, I ensured consistency across all pages.

For interactivity and functionality, I created a JavaScript file called portfolio.js. This file controls dynamic behaviors such as user interactions, navigation effects, and any responsive features on the site.

I also created a folder named “images,” where I stored all the images used in the project, such as my profile picture and project screenshots. This helps keep the project organized and makes it easier to manage assets.

Overall, this file structure allows the project to be clean, scalable, and easy to navigate, both for development and for future updates.

Now, I will begin by opening my first file and walking you through the code step by step.

SECTION 1: Explaining index.html (Speak while showing your HTML file): Now I will walk you through my index.html file, which represents the homepage of my portfolio.

I start with the DOCTYPE declaration. This tells the browser that I am using HTML5, which is the modern standard for building web pages.

Next, I open the HTML tag and set the language to English using the lang attribute. This helps with accessibility and search engine optimization.

Inside the head section, I define important metadata. I set the character encoding to UTF-8, which ensures that text displays correctly across different browsers.

I also include the viewport meta tag. This is very important because it makes my website responsive by controlling how it scales on different screen sizes, especially mobile devices.

Then I define the title of the page, which appears in the browser tab as “Edutech Portfolio.”

Next, I link my external CSS file called portfolio.css. This is where all my styling is defined, allowing me to separate structure from design.

I also connect to Google Fonts to import fonts like Montserrat, Poppins, and Roboto. This improves the visual appearance of my website.

Additionally, I include Font Awesome using a CDN. This allows me to use icons like the hamburger menu and highlight icons throughout the site.

Moving to the body section, I start with the header. Inside the header, I created a navigation bar using the nav element.

Inside the nav, I included a logo image using the img tag. The src points to my images folder, and I added an alt attribute for accessibility.

Next, I created a hamburger menu icon using a div with a Font Awesome icon inside it. This is mainly used for mobile responsiveness.

Then I created an unordered list with the class nav-links. This contains all my navigation links such as Home, About, Skills, Projects, Resume, and Contact. Each link connects to a different HTML page in my project.

The Home link has an active class, which helps visually indicate the current page.

After the header, I move to the main content area using the main tag.

Inside main, I created a section with the ID home-page. This section represents the hero section of my homepage.

Inside this section, I used a container div called home-container, and inside it another div called home-text.

Here, I added a heading that introduces what I do. I used a span inside the heading so I can style part of the text differently.

Below the heading, I added a paragraph describing my skills and what I build.

Then I added two buttons using anchor tags. One links to my projects page, and the other allows users to download my CV from the images folder.

After that, I created a highlights section. This contains three feature blocks.

Each highlight has an icon, a title, and a short description. The icons are from Font Awesome, which makes the design more modern and visually appealing.

Finally, I added a footer at the bottom of the page, which contains a simple copyright message.

At the very end of the body, I linked my JavaScript file portfolio.js. This ensures that all the HTML content loads first before JavaScript runs.

This completes the structure of my homepage using HTML.
SECTION 2: Explaining portfolio.css (Speak while showing CSS): Now I will explain how I styled my website using the portfolio.css file.

I started with a universal selector using the asterisk symbol. This resets margin and padding for all elements and sets box-sizing to border-box. This helps maintain consistent layout behavior across the site.

I also set a default font family to Roboto.

Next, I styled the body. I used flexbox with a column direction so that I can control layout easily, especially for pushing the footer to the bottom of the page.

I set the main element to flex 1, which allows it to expand and fill available space.

For the header, I gave it a black background and white text. I also added padding at the top to prevent content from hiding behind the fixed navigation bar.

For the navigation bar, I used flexbox to align items horizontally and space them evenly. I made the navbar fixed at the top so it stays visible while scrolling.

I added a gradient background, a border, and a box shadow to give it a modern look.

I styled the hamburger menu and initially set it to display none. Then, using a media query for screens smaller than 768 pixels, I made it visible.

Inside the same media query, I hid the navigation links by default and only display them when the active class is added. This is how the mobile menu works.

For the navigation links, I removed list styling and displayed them in a row using flexbox.

I styled links with white color and added a hover effect to change their color when users interact with them.

I also styled the logo image to maintain a proper size inside the navbar.

Next, I created reusable button styles called btn-red and btn-blue. These include padding, colors, border radius, and a hover effect using transform scale to make them slightly bigger when hovered.

For the homepage section, I set a dark background color and white text. I also styled the heading with a large font size, text alignment, and a shadow effect to make it visually strong.

I styled the paragraph to control width, alignment, and spacing.

I used flexbox again for the highlights container to arrange the cards neatly and make them responsive.

Each highlight card has a background color, padding, border radius, and glowing box shadow effect. I also added a hover effect to slightly lift the card and enhance the glow.

For responsiveness, I used media queries to reduce font sizes and adjust layout for smaller screens.

Finally, I implemented scroll animations. Initially, sections and project cards are hidden using opacity and transform.

When the visible class is added through JavaScript, they smoothly fade in and slide into view.

This completes the styling of my portfolio using CSS.
SECTION 3: Explaining portfolio.js (Speak while showing JS): Now I will explain how I added interactivity using JavaScript in my portfolio.js file.

I started by listening for the DOMContentLoaded event. This ensures that my JavaScript runs only after the HTML content has fully loaded.

Inside this function, I selected all sections and project cards using querySelectorAll.

Then I created a function called revealElements. This function checks the position of each element relative to the viewport.

If an element is within a certain distance from the screen, I add a class called visible to it.

This works together with my CSS animations to create a smooth scroll effect where elements appear as the user scrolls.

I added a scroll event listener to the window. Every time the user scrolls, the revealElements function runs and reveals content dynamically.

I also triggered the function once on page load so that visible elements appear immediately.

Next, I implemented contact form functionality using EmailJS.

I initialized EmailJS using my public key.

Then I selected the contact form and added a submit event listener.

When the form is submitted, I prevent the default behavior and send the form data using EmailJS.

If the message is sent successfully, I display an alert and reset the form. If there is an error, I display an error message.

Finally, I implemented the mobile navigation toggle.

I selected the hamburger icon and the navigation links.

When the hamburger icon is clicked, I toggle the active class on the nav links.

This allows the menu to appear and disappear on smaller screens.

This completes the interactivity and functionality of my portfolio application using JavaScript.

SECTION 1: Explaining about.html (Speak while scrolling your HTML):Now I will walk you through my about.html file, which contains the About page of my portfolio.

Just like my homepage, I begin with the DOCTYPE declaration to define the document as HTML5.

Next, I open the HTML tag and set the language to English, which improves accessibility and SEO.

Inside the head section, I define the meta tags. I set the character encoding to UTF-8 to ensure proper display of text, and I include the viewport meta tag to make the page responsive across different devices.

I then set the title of the page to “About - Edutech Portfolio,” which helps identify the page in the browser tab.

Next, I link my external stylesheet portfolio.css. This ensures that all styling is consistent across my website.

I also import Google Fonts to enhance typography and include Font Awesome so I can use icons where needed.

Moving into the body, I start with the header and navigation bar. This section is consistent across all pages to maintain a uniform user experience.

I include the logo image, the hamburger menu for mobile responsiveness, and the navigation links. The About link is marked as active to indicate the current page.

After the header, I define a section with the ID “about-page.” This is the main container for all the content on this page.

I begin with a heading “About Me,” which introduces the section.

Next, I create a container called about-container. Inside it, I divide the content into two parts: text and image.

In the about-text div, I include paragraphs that describe who I am, my skills, and my focus as a developer. I use the strong tag to emphasize important technologies like HTML, CSS, JavaScript, React, Python, and Django.

Beside the text, I include an image inside the about-image div. This displays a portrait or visual representation, making the page more engaging.

Below that, I created another section called about-extra. This highlights my core expertise using an unordered list. Each list item represents a key area of my skill set.

Next, I added a statistics section using about-stats. This includes three stat cards showing years of experience, completed projects, and learners impacted. These help provide quick, visual credibility.

After that, I created a timeline section called about-timeline. This shows my journey as a developer, from when I started coding to my current stage. Each item includes a year and a milestone.

Finally, I added a call-to-action section. This includes a button that allows users to download my resume.

At the bottom, I included a footer with a copyright message.

And just like before, I linked my JavaScript file at the end of the body to ensure all elements load before scripts run.

This completes the structure of my About page.
SECTION 2: Explaining About Page CSS: Now I will explain how I styled the About page using CSS.

I started by targeting the about-page ID. I set a minimum height of 100vh so that the page always fills the full screen. I also used a dark blue background and white text to maintain my design theme.

For the heading, I increased the font size, added spacing, and applied a strong text-shadow effect to make it visually appealing and prominent.

I styled the paragraph text to have a readable font size, controlled width, and proper alignment for better readability.

Next, I used flexbox for the about-container. This allows the text and image to sit side by side on larger screens and stack on smaller screens.

The about-text section is flexible and maintains good spacing and readability with line height adjustments.

For the image, I set a fixed width and height, applied object-fit to maintain proportions, and added a glowing box-shadow effect to enhance visual appeal.

In the about-extra section, I styled the heading and list items. I used underline and spacing to make the content clear and structured.

I also styled the strong tag inside paragraphs to highlight key technologies with a different color and underline.

For the stats section, I used flexbox again to align the stat cards in a row. Each card has padding, rounded corners, and a glowing shadow.

I added hover effects so that each stat card slightly lifts and changes its glow when the user hovers over it, making the UI interactive.

Next, I styled the timeline section. I removed default list styles and positioned the year using absolute positioning, which creates a clean timeline effect.

For the call-to-action button, I used a bright color, padding, and hover transition to make it stand out and encourage interaction.

Finally, I added responsive design using media queries. On smaller screens, I stack the layout vertically, reduce font sizes, and adjust image sizes to ensure the page remains user-friendly.

This ensures the About page looks good on both desktop and mobile devices.
SECTION 3: Explaining JavaScript & Its Impact on the About Page: Now I will explain how JavaScript improves the functionality and user experience of the About page.

Although the JavaScript file is shared across all pages, it plays an important role in enhancing this page.

First, I used the DOMContentLoaded event to ensure that all HTML elements are fully loaded before any JavaScript runs.

Then, I selected all section elements on the page. This includes the about-page section.

I created a function called revealElements. This function checks if elements are visible within the user's screen using getBoundingClientRect.

When a section enters the viewport, the function adds a class called visible.

This works together with my CSS animation. Initially, sections are hidden with low opacity and shifted slightly down.

When the visible class is added, the section smoothly fades in and moves into place.

This creates a modern scroll animation effect on the About page, making the content feel more dynamic and engaging instead of static.

Next, I added a scroll event listener. This ensures that as the user scrolls through the About page, each section appears at the right time.

I also triggered the animation once when the page loads, so visible content appears immediately.

Another important feature is the hamburger menu functionality.

I selected the hamburger icon and navigation links. When the user clicks the hamburger icon, the active class is toggled.

This allows the navigation menu to open and close on smaller screens, improving mobile usability on the About page.

Additionally, I included EmailJS functionality. While it is mainly used on the Contact page, including it here ensures the script is reusable across all pages without errors.

Overall, JavaScript improves the About page by adding smooth animations, enhancing navigation, and making the user experience more interactive and responsive.

This makes the page feel more professional and engaging.
SECTION 1: Explaining skills.html (Speak while showing HTML): Now I will walk you through my skills.html file, which represents the Skills page of my portfolio.

Just like the other pages, I start with the DOCTYPE declaration to define the document as HTML5.

I then open the HTML tag and set the language to English to improve accessibility and SEO.

Inside the head section, I include the meta charset set to UTF-8 for proper text rendering, and the viewport meta tag to ensure the page is responsive across different screen sizes.

I set the title of the page to “My Skills - Edutech Portfolio,” which appears in the browser tab.

Next, I link my external stylesheet portfolio.css. This ensures consistent styling across all pages.

I also import Google Fonts to improve typography, and include Font Awesome so I can use icons for visual enhancement.

Moving to the body, I begin with the header and navigation bar. This structure is consistent across all pages to maintain a uniform design.

The Skills link is marked as active, which helps users know the current page they are on.

After the header, I create a section with the ID “skills-page.” This acts as the main container for all the skills content.

I start with a heading titled “My Skills,” followed by a paragraph with the class skills-intro. This paragraph briefly explains my expertise and the technologies I use.

Next, I created a skills grid to organize my skills into categories.

Inside this grid, I divided my skills into four main categories: Front-End Development, Back-End Development, Tools and Workflow, and Soft Skills.

Each category is placed inside a div with the class skills-category.

Inside each category, I used an unordered list to display the individual skills.

For the soft skills section, I included icons using Font Awesome to make the content more visually engaging.

After listing my skills, I added a call-to-action section. This includes a button that allows users to download my full resume.

Next, I created a proficiency section using skills-progress.

Inside this section, I used skill bars to visually represent my proficiency levels in different technologies like HTML, CSS, JavaScript, React, and Python with Django.

Each skill bar consists of a label and a progress bar. The fill inside the bar is controlled using inline styles that define the width percentage.

Finally, I added a footer and linked my JavaScript file at the bottom of the page.

This completes the structure of my Skills page.
SECTION 2: Explaining Skills Page CSS: Now I will explain how I styled the Skills page using CSS.

I started by targeting the skills-page ID. I set a minimum height of 100vh so the page fills the full screen, and I used a dark background with white text to maintain consistency with my design theme.

For the heading, I increased the font size, applied uppercase transformation, and added a strong text-shadow effect to make it stand out.

I styled the skills-intro paragraph by controlling its width, centering it, and adjusting the font size for readability.

Next, I created a flexible layout using the skills-grid class.

I used flexbox with wrapping enabled so that skill categories automatically adjust depending on screen size.

Each skills-category acts like a card. I gave it padding, a background color, rounded corners, and a glowing box-shadow effect.

I also added a hover effect so that when users interact with a card, it slightly moves upward and changes its background color.

Inside each category, I styled the headings and list items. I removed default list styling and aligned items properly.

For icons in the soft skills section, I added spacing and color to improve visibility.

Next, I styled the proficiency bars.

Each skill-bar contains a label and a bar container. The bar has a light background, while the fill inside it has a bright color.

The width of the fill represents the proficiency level, and I added a transition effect so the bar fills smoothly when it appears.

For the call-to-action section, I styled the button with padding, background color, and hover effects to make it interactive.

Finally, I added responsive design using media queries.

On smaller screens, I reduced font sizes and adjusted layout widths so that the content stacks properly and remains user-friendly.

This ensures the Skills page is fully responsive and visually appealing across devices.
SECTION 3: Explaining JavaScript & Its Impact on Skills Page: Now I will explain how JavaScript enhances the Skills page.

First, I used the DOMContentLoaded event to ensure all elements are fully loaded before executing any JavaScript code.

Then I selected all section elements, including the skills-page section.

I created a function called revealElements, which checks if elements are visible within the viewport.

When a section becomes visible, the function adds a class called visible.

This works with my CSS animation to create a smooth fade-in and slide-up effect when users scroll through the Skills page.

This makes the content more engaging and avoids a static appearance.

For example, when users scroll down to view the skills categories or progress bars, they appear gradually instead of loading all at once.

I also added a scroll event listener to continuously check when elements enter the screen.

Next, I implemented the hamburger menu functionality.

When the hamburger icon is clicked, it toggles the active class on the navigation menu.

This allows users on smaller screens to open and close the menu easily, improving mobile usability.

Additionally, although EmailJS is mainly used for the Contact page, including it here ensures the JavaScript file works consistently across all pages without errors.

Overall, JavaScript improves the Skills page by adding smooth animations, enhancing navigation, and creating a more interactive user experience.

This makes the page feel modern, dynamic, and professional.
SECTION 1: Explaining projects.html (Speak while showing HTML): Now I will walk you through my projects.html file, which is one of the most important parts of my portfolio because it showcases my real-world work.

I start with the DOCTYPE declaration to define the document as HTML5, followed by the HTML tag with the language set to English.

Inside the head section, I include the meta charset and viewport settings to ensure proper rendering and responsiveness across devices.

I then set the page title to “Projects - Edutech Portfolio.”

Next, I link my external CSS file portfolio.css, which handles all styling for this page.

I also import Google Fonts to enhance typography and include Font Awesome for icons.

Moving into the body, I begin with the header and navigation bar, which is consistent across all pages. The Projects link is marked as active to indicate the current page.

After that, I define a section with the ID “projects-page.” This serves as the main container for all project-related content.

I start with a heading titled “My Projects,” followed by a short introduction paragraph explaining what the section represents.

Next, I created a project grid using a div with the class project-grid. This grid layout allows me to display multiple projects in a structured and responsive way.

Inside this grid, I created individual project cards using the class project-card.

Each project card contains several elements.

First, I added an image wrapped inside an anchor tag. This allows users to click the image and view it in full size.

Then I added a project title using the h3 tag.

Below that, I included a description paragraph explaining what the project does and its purpose.

I also added a list of features using an unordered list. This highlights key functionalities of each project.

Next, I included a tech stack section, where I display the technologies used in the project as small badges.

Finally, I added two buttons: one for viewing the source code on GitHub and another for viewing the live demo.

I repeated this structure for each project, including my portfolio website, calculator app, and boutique thrift application.

After the project grid, I added a testimonials section.

This includes feedback from users or reviewers, displayed using blockquote elements. This helps build credibility and trust.

At the bottom, I included a footer and linked my JavaScript file.

This completes the structure of my Projects page.
SECTION 2: Explaining Projects Page CSS: Now I will explain how I styled the Projects page using CSS.

I started by targeting the projects-page ID. I set a minimum height of 100vh so the page fills the full screen, and I used a dark background with white text to maintain consistency.

For the heading, I increased the font size, applied uppercase styling, and added a strong text-shadow effect to make it visually impactful.

I styled the introduction paragraph by controlling its width, centering it, and adjusting the font size for readability.

Next, I created the project grid using CSS Grid.

I used grid-template-columns with auto-fit and minmax. This allows the layout to automatically adjust the number of columns depending on screen size, making it fully responsive.

Each project-card is styled as a card with padding, rounded corners, and a shadow effect.

I added a hover effect so that when users move their cursor over a card, it slightly lifts upward, creating a modern interactive feel.

For images, I set the width to 100% and added border-radius to ensure they fit nicely inside the card.

I styled the project titles and descriptions with proper spacing and colors for readability.

For the feature list, I added padding and spacing between list items to make them easy to scan.

The tech stack is displayed using small badge-like elements. Each span has a background color, padding, and rounded edges to make it stand out.

For the buttons, I styled them with a bright background color, padding, and hover effects to make them interactive and noticeable.

Finally, I styled the testimonials section using a semi-transparent background and a left border to give it a quote-style appearance.

I also added responsive adjustments using media queries to ensure the layout works well on smaller screens.

This ensures the Projects page is visually appealing, structured, and responsive.

SECTION 3: Explaining JavaScript & Its Impact on Projects Page: Now I will explain how JavaScript enhances the Projects page.

This page benefits the most from JavaScript because of the animations applied to project cards.

First, I used the DOMContentLoaded event to ensure all elements are loaded before the script runs.

Then I selected all sections and all project cards using querySelectorAll.

I created a function called revealElements, which checks whether elements are visible within the viewport.

When a project card enters the visible area of the screen, the function adds a class called visible.

This works together with my CSS to animate the cards.

Initially, project cards are slightly hidden and shifted downward.

When the visible class is added, they smoothly fade in and move upward into position.

This creates a professional scroll animation effect, making the project showcase more engaging and visually appealing.

I also added a scroll event listener so that this animation triggers as the user scrolls through the page.

This ensures each project card appears at the right time instead of loading all at once.

Next, I implemented the hamburger menu functionality.

When the hamburger icon is clicked, it toggles the navigation menu on smaller screens, improving usability on mobile devices.

Additionally, I included EmailJS functionality for consistency across all pages, even though it is mainly used on the Contact page.

Overall, JavaScript enhances the Projects page by making it interactive, dynamic, and visually engaging, especially through scroll animations and responsive navigation.

This helps create a better user experience and makes my work stand out more effectively.
SECTION 1: Explaining resume.html (Speak while showing HTML): Now I will walk you through my resume.html file, which represents the Resume page of my portfolio.

Just like the other pages, I start with the DOCTYPE declaration to define the document as HTML5.

I then open the HTML tag and set the language to English to improve accessibility and search engine optimization.

Inside the head section, I include the meta charset set to UTF-8 to ensure proper text display, and the viewport meta tag to make the page responsive on different devices.

I set the title of the page to “Resume - Edutech Portfolio,” which appears in the browser tab.

Next, I link my external CSS file portfolio.css for styling, and I also import Google Fonts and Font Awesome to enhance the visual appearance and allow the use of icons.

Moving into the body, I start with the header and navigation bar, which is consistent across all pages. The Resume link is marked as active to indicate the current page.

After that, I define a section with the ID “resume-page.” This serves as the main container for all resume-related content.

I begin with a heading titled “Resume,” followed by an introduction paragraph explaining what the section contains.

Below that, I added a button that allows users to download my CV directly. This improves accessibility for recruiters who want a full version of my resume.

Next, I created a contact section using a div with the class resume-contact.

Here, I included my email, phone number, and LinkedIn profile. I used icons from Font Awesome to make each contact item visually clear and professional.

I also used anchor tags so that users can click to send an email, make a call, or visit my LinkedIn profile.

After that, I structured the rest of the page into different resume sections.

I created separate sections for Education, Training, Certifications, Experience, and Key Skills.

Each section is wrapped in a div with the class resume-section, and contains a heading and a list of items.

For the Experience section, I used a nested structure with a class called job.

Inside it, I included the job title, duration, and a short description of my responsibilities and technologies used.

Finally, I added a footer and linked my JavaScript file at the bottom of the page.

This completes the structure of my Resume page.
SECTION 2: Explaining Resume Page CSS:Now I will explain how I styled the Resume page using CSS.

I started by targeting the resume-page ID. I set a minimum height of 100vh so the page fills the screen, and I used a dark background with white text to maintain consistency across the website.

For the heading, I increased the font size, applied uppercase styling, and added a strong text-shadow effect to make it visually prominent.

I styled the introduction paragraph by controlling its width, centering it, and adjusting the font size for readability.

Next, I styled the resume-contact section.

I aligned the text to the left and added spacing between items. I also styled the icons and links to make them clear and interactive.

For the resume-section class, I created a structured layout by controlling width, spacing, and alignment.

Each section heading is styled with padding, background color, border, and rounded corners to make it stand out clearly from the rest of the content.

I removed default list styling and adjusted spacing for list items to improve readability.

For the experience section, I styled the job title with a highlight color and formatted the job date to appear slightly smaller and lighter.

I also created a reusable button style using the btn class.

This includes padding, background color, rounded corners, and a hover effect where the button slightly enlarges and changes color.

Finally, I added responsive design using media queries.

On smaller screens, I reduced widths, font sizes, and adjusted layout so that everything stacks properly and remains easy to read.

This ensures the Resume page is clean, professional, and accessible on all devices.
SECTION 3: Explaining JavaScript & Its Impact on Resume Page: Now I will explain how JavaScript enhances the Resume page.

Even though the Resume page is mostly informational, JavaScript still plays an important role in improving the user experience.

First, I used the DOMContentLoaded event to ensure that all elements are fully loaded before the script runs.

Then I selected all section elements, including the resume-page section.

I created a function called revealElements, which checks whether elements are visible within the viewport.

When the Resume section enters the screen, the function adds a class called visible.

This works with my CSS animation to create a smooth fade-in and slide-up effect when the user scrolls.

This makes the Resume page feel more modern and dynamic instead of static.

Next, I added a scroll event listener so that the animation triggers as the user scrolls through the page.

I also implemented the hamburger menu functionality.

When the hamburger icon is clicked, it toggles the navigation menu, making it easier to navigate on smaller screens.

Additionally, I included EmailJS functionality for consistency across all pages, even though it is mainly used on the Contact page.

Overall, JavaScript enhances the Resume page by adding smooth animations, improving navigation, and ensuring a consistent interactive experience across the entire portfolio.

This helps present my professional information in a more engaging and polished way.
CONTACT PAGE PRESENTATION SCRIPT: Now I will be explaining my Contact Page, which is designed to allow users, recruiters, or collaborators to easily reach out to me.

This page combines HTML for structure, CSS for styling and layout, and JavaScript for interactivity—especially for sending messages using EmailJS.

HTML EXPLANATION (Structure of the Contact Page): Now I will be explaining the HTML structure of my Contact Page.

I started with the standard HTML5 structure using the <!DOCTYPE html> declaration, which ensures the browser renders the page correctly.

Inside the <html> tag, I set the language to English using lang="en" for accessibility and SEO.

In the <head> section, I included important elements such as:

* Meta charset for proper text encoding
* Viewport meta tag for responsiveness on mobile devices
* Page title (Contact - Edutech Portfolio)
* Link to my external stylesheet (portfolio.css)
* Google Fonts for improved typography
* Font Awesome for icons
* EmailJS CDN script for sending messages directly from the browser

This setup ensures the page is modern, responsive, and functional.
Header and Navigation: Inside the <body>, I created a header section that contains the navigation bar.

The navigation includes:

* A logo image
* A hamburger menu icon for mobile devices
* Navigation links to other pages like Home, About, Skills, Projects, Resume, and Contact

The "Contact" link has an "active" class, which visually highlights the current page.

This ensures consistent navigation across the entire website.
Main Contact Section: The main content is wrapped inside a with the ID "contact-page".

This section includes:

A main heading (Contact)
An introductory paragraph explaining how users can reach out

This helps guide the user and gives context to the page. 📃 Contact Form: Inside the contact container, I created a form card.

The form includes:

* Input field for Name

* Input field for Email
* Textarea for Message

Each input field has:

* A label for accessibility
* A placeholder for user guidance
* The "required" attribute to enforce validation

The form has a class called "contact-form", which is important because it is used in JavaScript to handle form submission.

Finally, I added a submit button labeled "Send Message".
🔐 Privacy Note: Below the form, I added a privacy note to reassure users that their information will be kept confidential.

This improves trust and professionalism.
📞 Contact Information Card: Next, I created a second card for contact information.

This includes:

* Email address
* Phone number
* Location

Each item uses Font Awesome icons to make the interface more visual and user-friendly.
🌐 Social Links:I added social media links such as:

* GitHub
* LinkedIn
* Twitter

Each link opens in a new tab using target="_blank", allowing users to explore my profiles without leaving the site.
🗺️ Map Embed: I embedded a Google Map using an iframe.

This visually shows my location and adds credibility to the portfolio.

The map is responsive and styled using CSS.
CSS EXPLANATION (Styling and Layout): 🌌 Page Design: The contact page uses a dark navy background (#002347) with white text.

This creates a professional and modern design that is consistent across all pages of the portfolio.

🧱 Layout with Flexbox: I used Flexbox to structure the layout of the page.

The contact container uses:

display: flex;
flex-wrap: wrap;

This creates a two-column layout:

* One side for the form
* One side for contact details

On smaller screens, it automatically stacks into a single column.
🧾 Form Styling: The form inputs are styled to improve usability:

* Full width for easy typing
* Padding for spacing
* Rounded corners for a modern look

When a user focuses on an input field, I added a highlight effect using:

outline: 2px solid #ee5604;

This improves accessibility and user experience.
🔘 Button Styling: The submit button is styled with:

* Orange background (#ee5604)
* White text
* Rounded corners

On hover:

* The color darkens slightly
* Opacity changes

This gives users visual feedback when interacting with the button.
📦 Card Design: Both the form and contact info are placed inside cards.

These cards use:

* Light transparent backgrounds
* Rounded corners
* Box shadows

This creates a clean and modern UI appearance.
🌐 Social Icons: The social icons are styled to be larger and interactive.

When hovered:

* Their color changes

This improves engagement and usability.
📱 Responsive Design: Using media queries, I ensured that:

* The layout becomes vertical on small screens
* Text and elements resize properly

This guarantees a smooth experience across devices.
JAVASCRIPT EXPLANATION (Functionality): 🎬 Scroll Animations:I implemented scroll-based animations.

The script checks when elements enter the viewport and adds a "visible" class.

This creates smooth fade-in effects as the user scrolls through the page.
📩 EmailJS Form Handling:The most important functionality on this page is the contact form submission.

First, I initialize EmailJS using:

```
emailjs.init("your_user_id");
```

Then, I select the form using:

```
document.querySelector(".contact-form");
```

When the user submits the form:

* The default browser behavior is prevented
* The form data is sent using emailjs.sendForm()

If the message is sent successfully:

* A success alert is displayed
* The form resets

If there is an error:

* An error alert is shown

This allows real-time communication without needing a backend server.
🍔 Hamburger Menu: For mobile responsiveness, I implemented a hamburger menu.

When the icon is clicked:

* It toggles the "active" class on the navigation menu
* This shows or hides the menu

This improves usability on smaller devices.
🚀 FINAL PRESENTATION SUMMARY: In summary, the Contact Page is fully functional and interactive.

The HTML provides structure, the CSS enhances the design and responsiveness, and the JavaScript adds powerful features like animations and email functionality.

This page demonstrates my ability to build real-world, user-friendly web applications with modern tools and best practices.